SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering

1.Machine Learning Techniques for Reverse Engineering

Description:
Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:
Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

2. AI for Source Code Analysis and Program Understanding

Description:
AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:
Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

3. AI-Based Malware Reverse Engineering and Analysis

Description:
AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples**.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:
Students understand and evaluate how AI can support automated malware analysis.

4. Reverse Engineering of APIs Using AI

Description:
APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.
- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:
Students assess the usefulness of AI for API understanding and documentation recovery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Reverse Engineering Activity

AI for Source Code Analysis and Program Understanding

1. Objective

To use an AI-based approach to analyze an existing software project, understand its source code structure, identify important functions and dependencies, and verify the generated explanation with the actual code.

---

2. Selected Project

Project: A simple Python To-Do List application.

The project contains modules for:

- Adding tasks
- Viewing tasks
- Removing tasks
- Storing and managing task data

The purpose of the analysis is to understand the functionality of the program without starting from its original design documentation.

---

3. Reverse Engineering Process

The source code is analyzed to identify:

1. Program structure
2. Important functions
3. Input and output flow
4. Dependencies between components
5. Overall functionality

Basic Workflow

Existing Source Code

↓

AI Code Analysis

↓

Function Identification

↓

Dependency Analysis

↓

Generate Explanation

↓

Verify with Source Code

↓

Program Understanding

---

4. Source Code Used

```
tasks = []

def add_task(task):
    tasks.append(task)
    return "Task added successfully"

def view_tasks():
    return tasks

def remove_task(task):
    if task in tasks:
```

```
        tasks.remove(task)
        return "Task removed successfully"
    return "Task not found"
```

---

5. AI-Based Source Code Analysis

An AI code analysis tool was used to examine the program.

Function Analysis

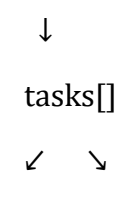
Function	Purpose
add_task(task)	Adds a new task to the task list.
view_tasks()	Displays or returns all available tasks.
remove_task(task)	Removes a specified task if it exists.

Data Dependency

All functions interact with the shared variable:

tasks

The dependency flow is:



The AI-generated explanation was compared with the actual source code.

---

6. Verification

The AI explanation was verified manually.

- add_task() correctly adds elements using append().
- view_tasks() returns the current task list.
- remove_task() checks whether a task exists before removing it.
- All three functions depend on the shared tasks list.

Therefore, the AI-generated analysis correctly represented the basic functionality and relationships within the program.

---

7. Benefits of AI in Reverse Engineering

AI-assisted source code analysis can help developers:

- Understand unfamiliar or legacy code quickly.
- Identify functions and dependencies.
- Generate documentation automatically.
- Detect relationships between software components.
- Reduce the time required for manual code analysis.

---

8. Limitations

AI-generated explanations should still be verified because:

- The explanation may misunderstand complex logic.
- Hidden dependencies may be missed.
- Generated documentation may not always reflect the exact implementation.

Therefore, **human verification remains important**.

---

9. Conclusion

The reverse engineering activity demonstrated how AI can assist in understanding an existing software system. By analyzing the source code, the important functions, data dependencies, and program flow were identified. The AI-generated explanation was then verified against the original code. Thus, AI can make reverse engineering faster and more efficient while still requiring human validation.